



Universidad Autónoma de Baja California
Facultad de Ingeniería Mexicali
Ingeniería en Computación

Reporte de Proyecto Final

**MASM x86 y Lenguaje de Alto Nivel:
Billar 8-ball**

Materia:

Organización y Arquitectura
de Computadoras

Profesor:

Dr. Omar Muñoz Urías

Integrantes del equipo:

Moya Monreal Erick Anselmo — 1110604
Martínez Zavala Alexandra — 1178815

Fecha de entrega

04/JUNIO/2026

Mexicali, Baja California
4 de junio de 2026

Índice

1. Introducción	2
2. Problemática y Justificación	2
3. Objetivos del proyecto	3
4. Marco Teórico	3
4.1. Programación Híbrida e Interoperabilidad C#/Ensamblador	3
4.2. La Unidad de Punto Flotante x87 (FPU)	4
4.3. Geometría de la Mesa y Elementos del Juego	5
4.4. Modelo Físico: Movimiento, Fricción y Restitución	6
4.5. Colisiones Elásticas y Descomposición Vectorial	6
4.6. Reglas del 8-ball como Máquina de Estados	6
5. Procedimiento	6
6. Resultados con pruebas	8
6.1. Arquitectura general del sistema	8
6.2. Estructuras de datos compartidas	9
6.3. Aplicación del tiro: trigonometría con la FPU	10
6.4. Movimiento y fricción por cuadro	10
6.5. Colisiones elásticas entre bolas	11
6.6. Cálculo del tiro en ensamblador: <code>ComputeShot</code>	12
6.7. Reglas del 8-ball y máquina de estados	13
6.8. Validación de la colocación de la blanca	14
6.9. Pruebas de ejecución	15
6.10. Resumen de la división de responsabilidades	16
7. Conclusiones	17
Referencias	18
A. Repositorio de código	18

1. Introducción

El desarrollo de software de calidad rara vez ocurre en un solo nivel de abstracción; en la práctica profesional es frecuente combinar un lenguaje de alto nivel, que aporta productividad e interfaces ricas, con lenguaje ensamblador, que otorga control absoluto sobre los registros, la memoria y la unidad de punto flotante del procesador. Esta cooperación permite delegar la lógica crítica a rutinas de bajo nivel mientras la capa de presentación se mantiene clara y mantenible.

Este proyecto lleva esa idea a un juego de billar (*8-ball*) en el que la **totalidad de la lógica y la física** reside en lenguaje ensamblador MASM x86, compilado como una biblioteca de enlace dinámico (`BillarLogica.dll`), mientras que la interfaz gráfica, escrita en C# con *Windows Presentation Foundation* (WPF), se limita estrictamente a dibujar la mesa y a capturar el ratón. El motor en ensamblador integra el movimiento de las bolas, aplica fricción, resuelve las colisiones elásticas entre bolas y contra las bandas, detecta los embocados en las troneras, predice la trayectoria del tiro y arbitra las reglas completas del 8-ball. La interfaz nunca decide nada sobre el estado del juego, únicamente le pregunta a la DLL qué ocurrió en cada cuadro y vuelve a pintar el resultado.

2. Problemática y Justificación

El planteamiento fue enfrentarnos al reto de **combinar un lenguaje de bajo nivel con uno de alto nivel**, que toda la lógica del programa se escribiera en ensamblador (para ejercer un control preciso del flujo, de la memoria y de los recursos de la máquina) delegando únicamente la interfaz gráfica a un lenguaje de alto nivel. El tema fue libre; mi compañera y yo elegimos un simulador de billar porque nos pareció una excelente manera de demostrar el uso del ensamblador tanto para el **cálculo matemático** (trigonometría, vectores y colisiones) como para el **manejo explícito de la memoria** (estructuras compartidas, punteros y desplazamientos).

El proyecto se justifica como una oportunidad para aplicar los conceptos de organización y arquitectura de computadoras en un contexto práctico, integrando la teoría con la implementación real. Además, el desarrollo de un juego de billar 8-ball permite explorar una amplia gama de temas: desde la física del movimiento y las colisiones hasta la gestión de estados y reglas del juego, todo ello implementado en ensamblador. La interoperabilidad entre C# y ensamblador también es un aspecto crucial, ya que refleja escenarios reales donde se combinan lenguajes para aprovechar sus fortalezas respectivas.

3. Objetivos del proyecto

Objetivo General

Diseñar e implementar un programa que combine lenguaje ensamblador MASM x86 y un lenguaje de alto nivel (C#/WPF) para crear un simulador de billar 8-ball híbrido en el que la lógica y la física completas residan en lenguaje ensamblador MASM x86 expuesto como una DLL, y la interfaz gráfica en C#/WPF actúe únicamente como capa de presentación y captura de entrada, demostrando la interoperabilidad entre ambos lenguajes.

Objetivos Específicos

- **Separar y confinar** la totalidad de la lógica (física, colisiones, reglas y predicción) al motor en ensamblador, reservando para C# exclusivamente el renderizado y la lectura del ratón.
- **Implementar** la integración del movimiento y la fricción de las bolas en cada cuadro empleando la pila de la FPU y la aritmética de doble precisión.
- **Resolver** las colisiones elásticas entre bolas.
- **Calcular** en ensamblador el ángulo y la fuerza del tiro a partir del gesto del ratón, así como la predicción de la trayectoria antes de ejecutar el tiro.
- **Arbitrar** las reglas del 8-ball mediante una máquina de estados, evaluando faltas, asignación de grupos y la condición de victoria de forma independiente para cada jugador.
- **Exponer** el motor como una DLL con convención `stdcall` e invocarla desde C# mediante *Platform Invoke*, compartiendo estructuras con una disposición de memoria idéntica.

4. Marco Teórico

4.1. Programación Híbrida e Interoperabilidad C#/Ensamblador

La programación híbrida consiste en combinar dos lenguajes dentro de un mismo ejecutable, asignando a cada uno la tarea para la que resulta más adecuado. A diferencia de la combinación C / ensamblador en línea, donde el bloque `__asm` comparte el espacio de la función anfitriona, un proyecto en C# no admite ensamblador en línea, por lo que la cooperación se logra compilando el código MASM como una **biblioteca de enlace dinámico** (DLL) que exporta sus funciones, e invocándolas desde C# mediante *Platform Invoke* [1]. El atributo `[DllImport]` indica el nombre de la DLL y la convención de llamada, y el

motor de *marshalling* traduce los tipos entre el mundo administrado de .NET y el mundo nativo.

El módulo en ensamblador se declara con `.model flat, stdcall`, de modo que tanto sus procedimientos como las llamadas desde C# siguen la convención `stdcall`, en la que los argumentos se apilan de derecha a izquierda y es la **función llamada** quien libera la pila al retornar [2]. Las funciones que se desean visibles se enumeran en un archivo de definición (`.def`) y se marcan con `PUBLIC`, lo que permite que el enlazador genere la tabla de exportación de la DLL.

4.2. La Unidad de Punto Flotante x87 (FPU)

La física del billar opera con números reales: posiciones, velocidades, ángulos y distancias. El procesador x86 maneja estos valores en la **unidad de punto flotante x87**, que organiza sus ocho registros como una **pila** (`ST(0)` a `ST(7)`) en lugar de registros de propósito general [3]. La instrucción `FLD` empuja un operando de memoria a la cima de la pila, `FSTP` extrae la cima hacia memoria, y las operaciones aritméticas (`FADD`, `FSUB`, `FMUL`, `FDIV`) combinan la cima con otro operando. Cada valor se almacena como `REAL8`, el formato de doble precisión de 64 bits.

Una particularidad de la FPU es que las comparaciones no afectan directamente a las banderas del procesador: `FCOMP` compara y deja el resultado en la palabra de estado de la FPU, que debe transferirse con `FNSTSW AX` y volcarse a las banderas con `SAHF` antes de poder usar un salto condicional. Este idioma `FCOMP / FNSTSW / SAHF` se repite en todo el motor cada vez que se necesita decidir con base en un número real. Además, la x87 ofrece funciones en *hardware*, `FSQRT` para la raíz cuadrada, `FSINCOS` para obtener seno y coseno simultáneamente, y `FPATAN` para el arcotangente de dos argumentos, indispensables para la trigonometría del tiro.

La Figura 1 ilustra cómo evoluciona la pila de la FPU al preparar un tiro: cada operación empuja o consume valores en la cima, sin registros con nombre.

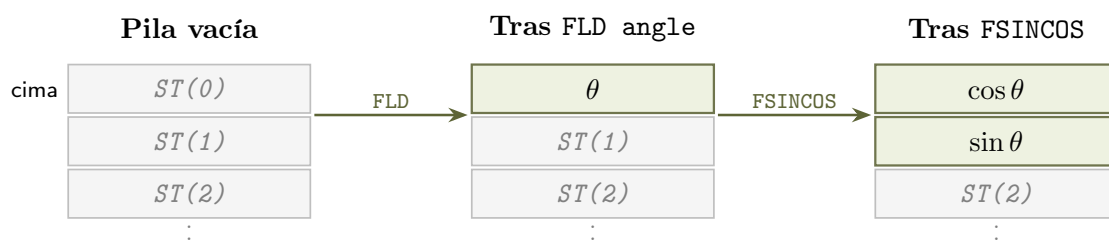


Figura 1: La pila de la FPU al preparar un tiro. `FLD` empuja el ángulo a la cima `ST(0)`; `FSINCOS` lo reemplaza por su coseno y deja el seno en `ST(1)`, ambos listos para escalar la velocidad de la blanca.

4.3. Geometría de la Mesa y Elementos del Juego

El motor trabaja sobre un sistema de coordenadas en píxeles, el área jugable es un rectángulo de 800×400 px (de 50 a 850 en X y de 50 a 450 en Y), con seis troneras de radio 22 px y dieciséis bolas de radio 14 px. Todas estas dimensiones viven como constantes **REAL8** en el motor y se entregan a la interfaz para que dibuje exactamente sobre las mismas coordenadas, evitando cualquier desfase entre la física y el render. La Figura 2 muestra la disposición de la mesa con el rack inicial y la bola blanca en la cabecera.



Figura 2: Mesa de billar y rack inicial. El motor usa coordenadas en píxeles (área jugable de 800×400 px); la bola blanca arranca en la cabecera y las 15 bolas numeradas forman el triángulo con la 8 al centro.

Cada bola se distingue por su **tipo**, almacenado como un entero en su estructura, la **blanca** (con la que se golpea), las **lisas** (números 1 a 7), las **rayadas** (9 a 15) y la **bola 8**. Ese tipo es el que el motor consulta para saber a qué jugador pertenece cada bola, como ilustra la Figura 3.



Figura 3: Los cuatro tipos de bola: blanca (taco), lisa (1–7), rayada (9–15) y la bola 8, que decide la partida.

4.4. Modelo Físico: Movimiento, Fricción y Restitución

El motor avanza la simulación en pasos discretos (un paso por cuadro, a unos 60 cuadros por segundo). En cada paso, la posición de cada bola se actualiza sumándole su velocidad, y a continuación la velocidad se atenúa multiplicándola por un **coeficiente de fricción** ligeramente menor que uno (0.985), lo que produce una desaceleración exponencial realista hasta que la bola se detiene al caer por debajo de un umbral mínimo [4]. Cuando una bola alcanza una banda, su componente de velocidad perpendicular a la pared se invierte y se multiplica por un **coeficiente de restitución** (0.95), modelando la pequeña pérdida de energía del rebote.

4.5. Colisiones Elásticas y Descomposición Vectorial

Cuando dos bolas de igual masa chocan, el intercambio de movimiento ocurre a lo largo de la **línea que une sus centros** (la dirección normal). La componente de la velocidad relativa sobre esa normal se transfiere a la bola golpeada, mientras que la componente **tangencial** (perpendicular a la normal) la conserva la bola que golpea. Este es el fundamento de la conocida “regla de los 90°”: tras un impacto no central, la blanca y la bola objeto se separan formando un ángulo recto. El motor detecta el choque cuando la distancia entre centros es menor o igual a un diámetro (28 px), calcula el vector normal unitario y proyecta sobre él las velocidades para realizar el intercambio [5]. La Figura 6 ilustra esta descomposición.

4.6. Reglas del 8-ball como Máquina de Estados

El flujo de una partida puede modelarse como una máquina de estados finita. La aplicación transita entre apuntar, cargar la fuerza, ejecutar el tiro, esperar a que todo se detenga y evaluar el resultado, momento en el que el motor decide si el jugador repite, cede el turno, comete falta (y entrega “bola en mano” al rival) o gana la partida [6].

5. Procedimiento

El desarrollo se organizó de forma incremental, partiendo de la definición de las estructuras compartidas y la comunicación entre ambos lenguajes, para avanzar después hacia la física, las colisiones y, finalmente, las reglas del juego. Las actividades realizadas fueron las siguientes:

1. **Definición de las estructuras compartidas.** Se declararon en ensamblador las estructuras `Ball`, `Pocket` y `GameState`, y su espejo exacto en C# con

[StructLayout(Sequential)], garantizando una disposición de memoria idéntica en ambos lados.

2. **Puente de interoperabilidad.** Se compiló el motor como `BillarLogica.dll` con convención `stdcall` y exportaciones en un archivo `.def`, y se declararon sus firmas en C# mediante `[DllImport]`.
3. **Inicialización y aplicación del tiro.** Se programó `InitGame` (colocación inicial de las 16 bolas) y `ApplyShot`, que convierte ángulo y fuerza en las componentes de velocidad de la blanca usando `FSINCOS`.
4. **Integración del movimiento y fricción.** Se implementó `UpdatePhysics`, que en cada cuadro actualiza posiciones, aplica la fricción y delega en las rutinas de colisión.
5. **Colisiones bola–bola y bola–banda.** Se desarrollaron `ResolveBallCollision` (colisión elástica con descomposición normal/tangencial) y `CheckWallCollisions` (reflexión con restitución).
6. **Troneras y predicción.** Se programó la detección de embocados y la rutina `PredictShot`, que lanza un rayo desde la blanca para anticipar el primer impacto, así como `ComputeShot`, que traduce el gesto del ratón en ángulo y fuerza.
7. **Reglas del 8-ball.** Se implementó `EvaluateTurn`, la máquina de estados que arbitra faltas, asignación de grupos, repetición de turno y victoria, de forma independiente por jugador.
8. **Interfaz y pruebas.** Se construyó la GUI en C#/WPF (renderizado de la mesa, taco, predicción y marcador) y se ejecutaron pruebas de partida completas, incluyendo casos límite (scratch, falta por grupo incorrecto y victoria con la 8).

6. Resultados con pruebas

6.1. Arquitectura general del sistema

El sistema se organiza como un ciclo que se repite en cada cuadro de animación (≈ 60 veces por segundo). La interfaz captura la entrada del usuario, invoca al motor en ensamblador a través del puente P/Invoke, y este actualiza el estado físico que la GUI vuelve a leer para dibujar. La Figura 4 resume este flujo y la división de responsabilidades.

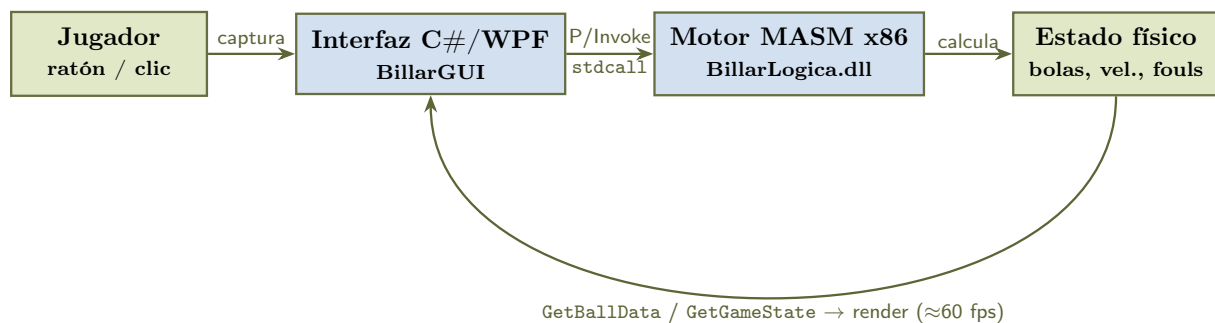


Figura 4: Arquitectura del sistema: la interfaz en C# solo dibuja y captura; el motor en ensamblador calcula toda la física y las reglas. Cada cuadro, la GUI consulta a la DLL y repinta.

El motor expone su funcionalidad mediante un conjunto de funciones, todas con convención `stdcall`, resumidas en la Tabla 1.

Tabla 1: Principales funciones exportadas por `BillarLogica.dll`.

Función	Responsabilidad
<code>InitGame</code>	Coloca las 16 bolas y reinicia el estado de la partida
<code>ApplyShot</code>	Convierte ángulo y fuerza en la velocidad de la blanca
<code>UpdatePhysics</code>	Un paso de simulación: mover, friccionar y colisionar
<code>EvaluateTurn</code>	Arbitra las reglas del 8-ball al detenerse las bolas
<code>PredictShot</code>	Anticipa el primer impacto del tiro (línea fantasma)
<code>ComputeShot</code>	Traduce el gesto del ratón en ángulo y fuerza
<code>PlaceCueBall</code>	Recoloca la blanca tras una falta (bola en mano)
<code>GetBallData / GetGameState</code>	Entregan a C# el estado para dibujar

6.2. Estructuras de datos compartidas

La base de la interoperabilidad es que ambos lenguajes vean los datos con la misma disposición en memoria. El Código 1 muestra la estructura `Ball` en ensamblador; su equivalente en C# se declara con `[StructLayout(LayoutKind.Sequential)]` para forzar el mismo orden y tamaño de campos.

```

1 Ball STRUCT
2     pos_x      REAL8    0.0      ; posicion X (pixeles)
3     pos_y      REAL8    0.0      ; posicion Y
4     vel_x      REAL8    0.0      ; velocidad X (px/cuadro)
5     vel_y      REAL8    0.0      ; velocidad Y
6     radius     REAL8    14.0     ; radio
7     ball_num   DWORD     0        ; 0=blanca, 1-7=lisas, 8=ocho,
8         9-15=rayadas
9     ball_type  DWORD     0        ; 0=blanca, 1=lisa, 2=rayada, 3=
10         ocho
11     is_active  DWORD     1        ; 1=en mesa, 0=embocada
12     was_pocketed DWORD    0      ; se emboco en este turno?
13 Ball ENDS

```

Listing 1: Estructura `Ball` en ensamblador (campos `REAL8` y `DWORD`).

La clave de la interoperabilidad es que cada campo ocupa un **desplazamiento** (*offset*) fijo desde la base de la estructura. La Figura 5 muestra esa disposición: cinco campos `REAL8` de 8 bytes seguidos de cuatro `DWORD` de 4 bytes, 56 bytes en total. El motor avanza de una bola a la siguiente sumando ese tamaño al puntero, y C# debe declarar la estructura idéntica para que ambos lados lean los mismos bytes.

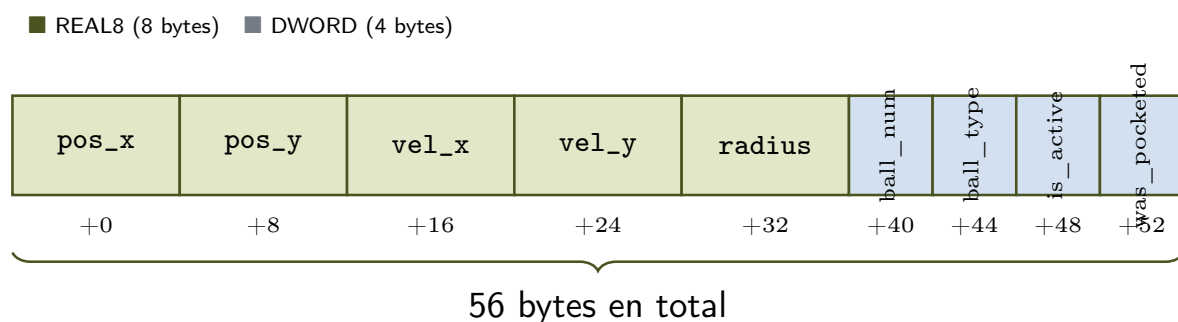


Figura 5: Disposición en memoria de la estructura `Ball` (56 bytes). Conocer el *offset* exacto de cada campo es lo que permite recorrer el arreglo de bolas con direccionamiento indirecto y compartir la estructura con C# sin que un solo byte se desalinee.

6.3. Aplicación del tiro: trigonometría con la FPU

La rutina ApplyShot recibe el ángulo y la fuerza del tiro y los convierte en las componentes vel_x y vel_y de la blanca. El Código 2 muestra el núcleo del cálculo, que descansa en la instrucción FSINCOS.

```

1      FLD [ESI].ShotInput.angle    ; ST(0) = angulo del tiro
2      FSINCOS                      ; ST(0)=cos, ST(1)=sin
3      FSTP temp_cos
4      FSTP temp_sin
5
6      FLD [ESI].ShotInput.power
7      FMUL MULT_SHOT               ; aplicar multiplicador global
8      FMUL MAX_SHOT_SPEED          ; escalar por la velocidad maxima
9      FLD ST(0)                    ; duplicar la magnitud
10     FMUL temp_cos                 ; vel_x = magnitud * cos(angulo)
11     FSTP [EDI].Ball.vel_x
12     FMUL temp_sin                 ; vel_y = magnitud * sin(angulo)
13     FSTP [EDI].Ball.vel_y

```

Listing 2: Conversión de ángulo y fuerza en velocidad con FSINCOS.

Análisis de instrucción

Instrucción

```

1      FLD [ESI].ShotInput.angle
2      FSINCOS

```

Descripción Carga el ángulo en la cima de la pila de la FPU y obtiene en una sola operación su coseno y su seno.

FSINCOS es una instrucción trascendente que sustituye dos llamadas separadas a seno y coseno. Tras ejecutarla, la cima ST(0) contiene el coseno y ST(1) el seno del ángulo original, que se guardan en sendas variables temporales. Multiplicando la magnitud del tiro (fuerza por la velocidad máxima) por el coseno se obtiene la componente horizontal de la velocidad, y por el seno la vertical. De esta forma, una operación que en C# sería Math.Cos y Math.Sin se resuelve aquí íntegramente en *hardware* de punto flotante.

6.4. Movimiento y fricción por cuadro

En cada llamada a UpdatePhysics, antes de evaluar colisiones, cada bola activa avanza según su velocidad y luego ve atenuada esa velocidad por el coeficiente de fricción. El Código 3 muestra el patrón aplicado al eje X (el eje Y es idéntico).

```

1      FLD [ESI].Ball.vel_x
2      FADD [ESI].Ball.pos_x        ; pos_x = pos_x + vel_x
3      FSTP [ESI].Ball.pos_x

```

```

4
5     FLD [ESI].Ball.vel_x
6     FMUL FRICITION_COEFF      ; vel_x = vel_x * 0.985
7     FSTP [ESI].Ball.vel_x

```

Listing 3: Integración de la posición y aplicación de la fricción.

6.5. Colisiones elásticas entre bolas

El corazón de la física es `ResolveBallCollision`, que intercambia el movimiento entre dos bolas a lo largo de la línea de sus centros. Primero se calcula el producto punto de la velocidad relativa con el vector normal unitario (dvn); si es positivo, las bolas se acercan y procede el intercambio; si no, solo se separan para evitar solapamientos. El Código 4 muestra el cálculo de dvn y la decisión.

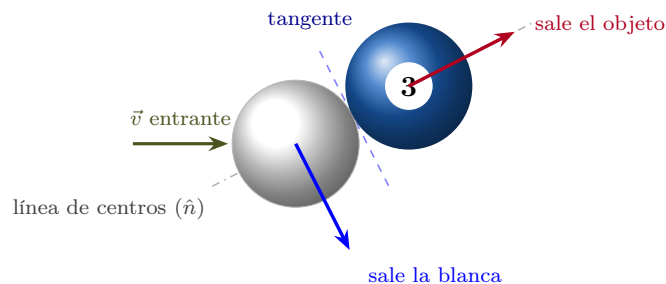


Figura 6: Colisión elástica de bolas de igual masa: la componente normal de la velocidad se transfiere a la bola objeto y la tangencial la conserva la blanca, separándose ambas en ángulo recto.

```

1     ; dvn = (v_i - v_j) . n = (vix-vjx)*nx + (viy-vjy)*ny
2     FLD [ESI].Ball.vel_x
3     FSUB [EDI].Ball.vel_x
4     FMUL temp_nx
5     FSTP temp_val
6     FLD [ESI].Ball.vel_y
7     FSUB [EDI].Ball.vel_y
8     FMUL temp_ny
9     FADD temp_val
10    FSTP temp_dvn      ; dvn guardado
11
12    FLD temp_dvn
13    FCOMP FP_ZERO      ; comparar dvn con 0
14    FNSTSW AX
15    SAHF
16    JBE do_overlap_only ; dvn <= 0: se separan, no
    intercambiar

```

Listing 4: Producto punto sobre la normal y decisión de intercambio en `ResolveBallCollision`.

Análisis de instrucción

Instrucción

```
1   FLD temp_dvn
2   FCOMP FP_ZERO
3   FNSTSW AX
4   SAHF
5   JBE do_overlap_only
```

Descripción Compara el valor real `dvn` contra cero trasladando el resultado de la FPU a las banderas del procesador para poder saltar.

Este fragmento ilustra el idioma de comparación de la FPU. `FCOMP` compara la cima de la pila con `FP_ZERO` y deja el resultado en la palabra de estado de la unidad de punto flotante, no en las banderas del procesador. Para poder usar un salto condicional, `FNSTSW AX` copia esa palabra de estado al registro `AX` y `SAHF` la transfiere a las banderas. Solo entonces `JBE` (*Jump if Below or Equal*) puede desviar el flujo cuando `dvn` es menor o igual a cero, caso en el que las bolas se están separando y no debe intercambiarse su velocidad. Esta secuencia de cuatro instrucciones es la traducción a bajo nivel de un simple `if (dvn <= 0)`.

6.6. Cálculo del tiro en ensamblador: `ComputeShot`

La rutina `ComputeShot` recibe las coordenadas de la blanca y del puntero y devuelve el ángulo, la fuerza y la dirección, sin que la interfaz realice trigonometría alguna. El Código 5 muestra el cálculo del ángulo con `FPATAN`.

```
1   FLD ptrY
2   FSUB cueY           ; ST(1) = dy = ptrY - cueY
3   FLD ptrX
4   FSUB cueX           ; ST(0) = dx = ptrX - cueX
5   FPATAN              ; ST(0) = atan2(dy, dx)
6   FSTP REAL8 PTR [EDI + 0] ; angulo del tiro
```

Listing 5: Cálculo del ángulo del tiro con `FPATAN` (equivalente a `atan2`).

Análisis de instrucción

Instrucción

1

FPATAN

Descripción Calcula el arcotangente del cociente $ST(1)/ST(0)$ usando el signo de ambos operandos para determinar el cuadrante correcto.

FPATAN resuelve el problema de obtener la dirección de un vector (dx, dy) . A diferencia de un arcotangente simple, considera el signo de las dos componentes para devolver un ángulo en el rango completo $(-\pi, \pi]$, igual que la función `atan2` de los lenguajes de alto nivel. Cargando primero dy y luego dx en la pila, la instrucción consume ambos y deja el ángulo en la cima, que se almacena en el búfer de salida que C# leerá para orientar el taco. Como el ángulo y las distancias son invariantes ante la traslación entre coordenadas de pantalla y de física, basta con pasar las coordenadas crudas del ratón.

6.7. Reglas del 8-ball y máquina de estados

La aplicación recorre una máquina de estados que la interfaz refleja visualmente. La Figura 7 muestra el ciclo de un turno: desde **Apuntar**, el jugador carga la fuerza, dispara, el motor simula hasta que todo se detiene y **Evaluar** decide el desenlace.

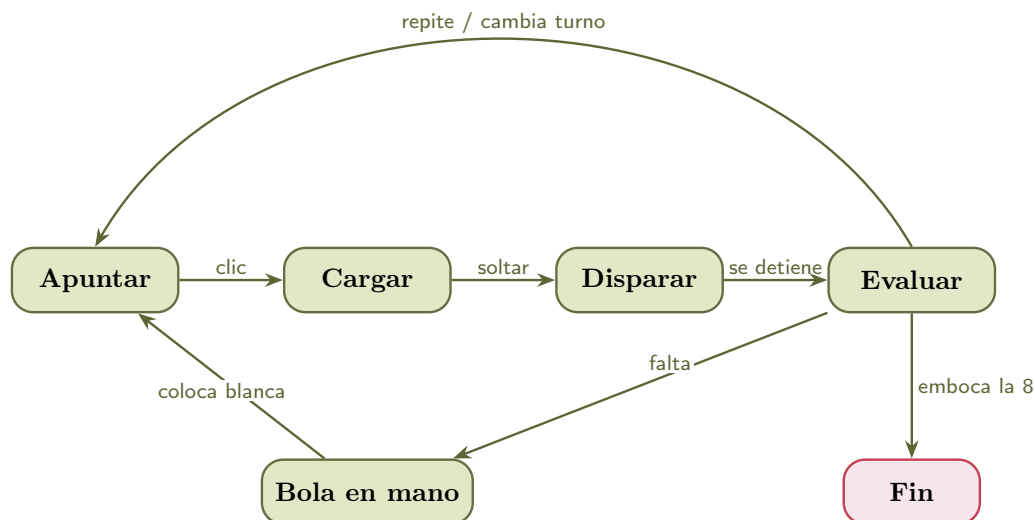


Figura 7: Máquina de estados de un turno (enum `AppState`). El estado **Disparar** corresponde a la simulación física; **Evaluar** es la llamada a `EvaluateTurn`.

La regla más delicada es que la obligación de jugar la bola 8 es **por jugador**. Al verificar una posible falta, el motor cuenta cuántas bolas del grupo del jugador en turno seguían en la mesa al **iniciar** el tiro —es decir, las que están activas o se embocaron en

este mismo turno— y, según ese conteo, exige que la primera bola golpeada sea de su grupo o la 8. El Código 6 muestra esa decisión.

```

1      ; EDX = bolas del grupo que seguían en mesa al iniciar el tiro
2      XOR EDX, EDX
3  fc_count_before:
4      CMP [ESI].Ball.ball_type, EBX      ; del grupo del jugador?
5      JNE fc_count_next
6      CMP [ESI].Ball.is_active, 1        ; sigue activa?
7      JE fc_count_inc
8      CMP [ESI].Ball.was_pocketed, 1     ; o se emboco este turno?
9      JNE fc_count_next
10 fc_count_inc:
11     INC EDX
12 fc_count_next:
13     ; ... avanzar al siguiente Ball ...
14
15     CMP EDX, 0
16     JG must_hit_own_group      ; aun tenia bolas -> debe golpear su
                                grupo
17     ; si EDX = 0, ya estaba en la 8 -> debe golpear la 8

```

Listing 6: Decisión por jugador: ¿debe golpear su grupo o la bola 8?

Análisis de instrucción

Instrucción

```

1      CMP EDX, 0
2      JG must_hit_own_group

```

Descripción Si quedaban bolas del grupo propio al iniciar el tiro, el jugador estaba obligado a golpear su grupo primero; de lo contrario, debía jugar la 8.

Contar las bolas que seguían en mesa **al iniciar** el tiro (incluyendo las embocadas en la jugada) evita un error concreto, embocar la última bola propia y, en el mismo tiro, tocar legalmente la 8 no debe penalizarse como si el jugador hubiera estado obligado a jugar la 8 desde antes. Esta evaluación independiente para cada jugador permite que, mientras uno ya va por la 8, el rival siga obligado a despejar su grupo, tal como dictan las reglas oficiales del 8-ball.

6.8. Validación de la colocación de la blanca

Tras una falta, el rival recibe “bola en mano”. La rutina PlaceCueBall valida que la posición elegida quede dentro del área jugable, sin penetrar las bandas, lejos de las troneras y sin solaparse con otra bola. El Código 7 muestra una de las comprobaciones de borde.

```

1      ; exigir new_x >= TABLE_LEFT + BALL_RADIUS
2      FLD new_x

```

```
3  FSUB TABLE_LEFT
4  FSUB BALL_RADIUS
5  FCOMP FP_ZERO
6  FNSTSW AX
7  SAHF
8  JB place_invalid      ; cae sobre el cojin -> posicion
   invalida
```

Listing 7: Validación del borde izquierdo en PlaceCueBall.

6.9. Pruebas de ejecución

Se realizaron partidas completas verificando tanto el juego normal como los casos límite. La Figura 8 muestra la interfaz durante el apuntado, con la línea de predicción y el taco; las Figuras 9a y 9b corresponden a una falta con bola en mano y a la victoria al embocar la 8.

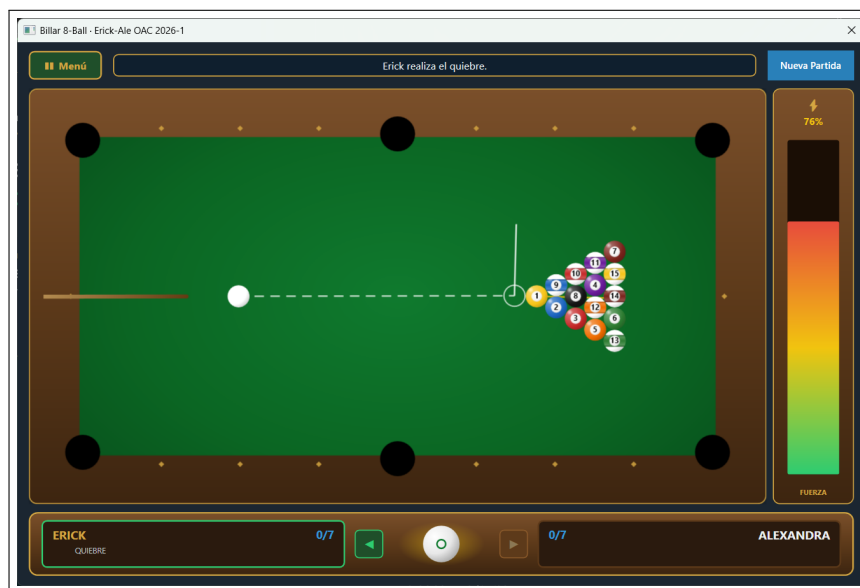


Figura 8: Apuntado del tiro: el motor predice el primer impacto y la GUI dibuja la línea fantasma, la bola guía y las líneas de salida.



(a) Falta: el rival recoloca la blanca.

(b) Victoria al embocar legalmente la bola 8.

Figura 9: Pruebas de los casos límite de las reglas del 8-ball.

6.10. Resumen de la división de responsabilidades

La Tabla 2 confirma que la totalidad de la lógica reside en ensamblador y que C# se limita a la presentación y la captura de entrada.

Tabla 2: División de responsabilidades entre el motor (MASM) y la interfaz (C#).

Tarea	Motor (MASM)	Interfaz (C#)
Física, fricción y colisiones	Sí	—
Reglas, faltas y turnos	Sí	—
Ángulo, fuerza y predicción del tiro	Sí	—
Dibujo de mesa, bolas y taco	—	Sí
Captura del ratón	—	Sí
Mostrar marcador y mensajes	—	Sí

7. Conclusiones

Desarrollar este proyecto nos permitió comprender, de una forma mucho más profunda que en las prácticas previas, lo que implica sostener un sistema interactivo en tiempo real con toda su lógica escrita en ensamblador. Confinar la física y las reglas al motor y reservar para C# únicamente el dibujo hizo evidente cuánto resuelve de manera implícita un lenguaje de alto nivel, la trigonometría del tiro, la raíz cuadrada de una distancia o la simple comparación de dos números reales tuvieron que expresarse explícitamente sobre la pila de la unidad de punto flotante, encadenando FLD, FSTP y el idioma FCOMP / FNSTSW / SAHF.

El manejo de la FPU fue el aprendizaje más valioso. Entender que sus registros funcionan como una pila y que cada FLD debe tener su correspondiente consumo nos obligó a llevar un control cuidadoso del balance de la pila en cada rutina; un valor olvidado en ST(0) corrompía silenciosamente la siguiente operación, lo que se convirtió en la fuente de error más difícil de rastrear.

La interoperabilidad entre C# y el ensamblador resultó más accesible de lo esperado una vez comprendida la convención `stdcall` y la importancia de que las estructuras compartidas tuvieran exactamente la misma disposición en memoria de ambos lados. Cualquier discrepancia en el tamaño de un campo se manifestaba como datos basura.

Referencias

- [1] Microsoft. “Platform Invoke (P/Invoke),” Microsoft Learn. dirección: <https://learn.microsoft.com/dotnet/standard/native-interop/pinvoke>.
- [2] K. R. Irvine, *Assembly Language for x86 Processors*, 8th. Hoboken, NJ: Pearson, 2020, ISBN: 978-0-13-538165-6.
- [3] Intel Corporation, “Intel 64 and IA-32 Architectures Software Developer’s Manual, Volume 1: Basic Architecture,” Intel Corporation, inf. téc. 253665, 2023.
- [4] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, ARM. Morgan Kaufmann, 2016.
- [5] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th. Hoboken, NJ: Pearson Education, Inc., 2019, ISBN: 978-0-13-499719-3.
- [6] A. S. Tanenbaum y T. Austin, *Structured Computer Organization*, 6th. Pearson, 2013.
- [7] K. R. Irvine, *Assembly Language for x86 Processors*, 6th. Upper Saddle River, NJ: Prentice Hall, 2011.

A. Repositorio de código

El código completo del proyecto híbrido (C#/WPF + **MASM x86**), junto con las instrucciones de compilación y ejecución, se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/ApoloEM/ApoloEM-OyAdC-ProyectoFinal2026-1>